Maintenance Management System (MMS)

Software Engineering Project Report

Group Number: 01
Course: Software Engineering
Semester: Spring 2026
Instructor: Dr. Samer Elkababji

Group Members

Student ID	Name	Commit Count
20210611	Osama	[count]
20210748	Mira	[count]
20210853	Zina	[count]
20220637	Zakarea	[count]

GitHub Repository URL:
https://github.com/xalameen8633-dotcom/Software-engineering-project

Version Information:
v1.0.0: Initial report version

---

1. Project Overview

The Maintenance Management System (MMS) is a database-based and sensor-enabled system that helps manage preventive, corrective and predictive maintenance tasks in industrial, commercial or institutional environments. The system enables technicians to log maintenance requests, supervisors to review and assign work orders, and administrators to monitor maintenance performance through dashboards and KPI reports.

Moreover, the system can connect to IoT sensors that track equipment conditions like temperature, vibration, and pressure. The system can issue predictive maintenance alerts and automatically submit maintenance requests when anomalies are detected. This allows businesses to minimize downtime, plan maintenance activities and enhance efficiency.

1.1 System Purpose

The MMS serves multiple stakeholders:

- **Technician:** Logs maintenance requests, views assigned work orders, updates work progress, and submits work order completion.
- **Supervisor:** Reviews maintenance requests, approves or rejects requests, assigns technicians, monitors work order progress, and confirms closure.

- **Administrator:** Monitors dashboards, KPIs, and maintenance reports.
- **IoT Sensors:** Send abnormal sensor readings to the system for predictive maintenance detection.
- **Notification Service:** Sends alerts and work order notifications to relevant users.

1.2 System Scope

The system covers the following functional areas:

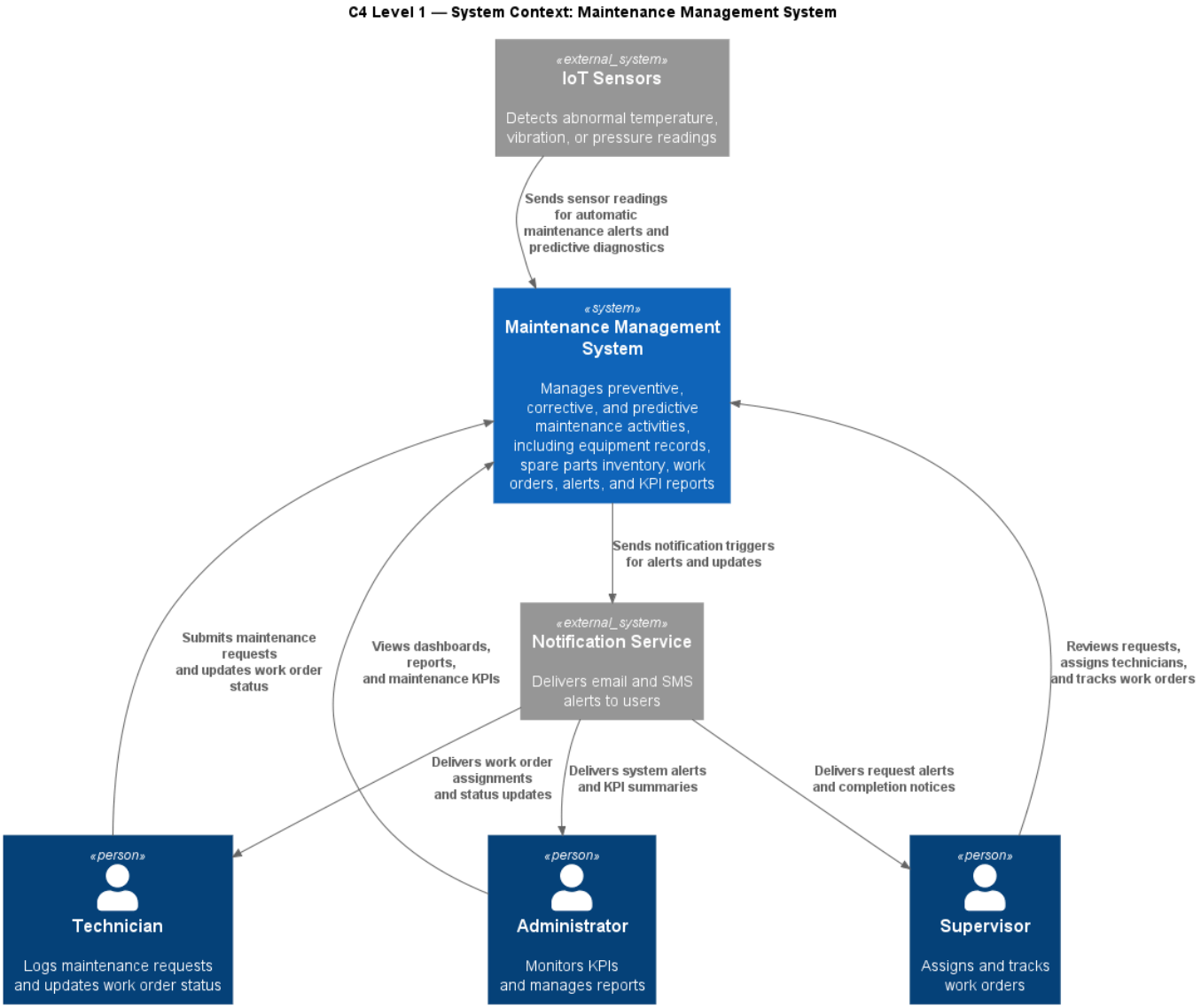
- Logging corrective maintenance requests
- Monitoring sensor readings for predictive maintenance
- Generating predictive alerts
- Automatically creating maintenance requests
- Approving and assigning work orders
- Tracking work order progress
- Recording labour and spare parts used
- Confirming and closing completed work orders
- Monitoring KPIs and dashboards
- Sending maintenance notifications

---

2. Context Diagrams and Activity Diagram

2.1 Context Model

The C4 Level 1 Context Diagram defines the Maintenance Management System boundary and illustrates its interactions with main users and external systems. All interacting entities are categorized according to their roles and responsibilities.



Internal Users

- **Technician:** Logs maintenance requests, views assigned work orders, and submits completion updates.
- **Supervisor:** Reviews requests, approves them, assigns work orders, tracks progress, and closes requests.
- **Administrator:** Monitors maintenance KPIs and generates reports.

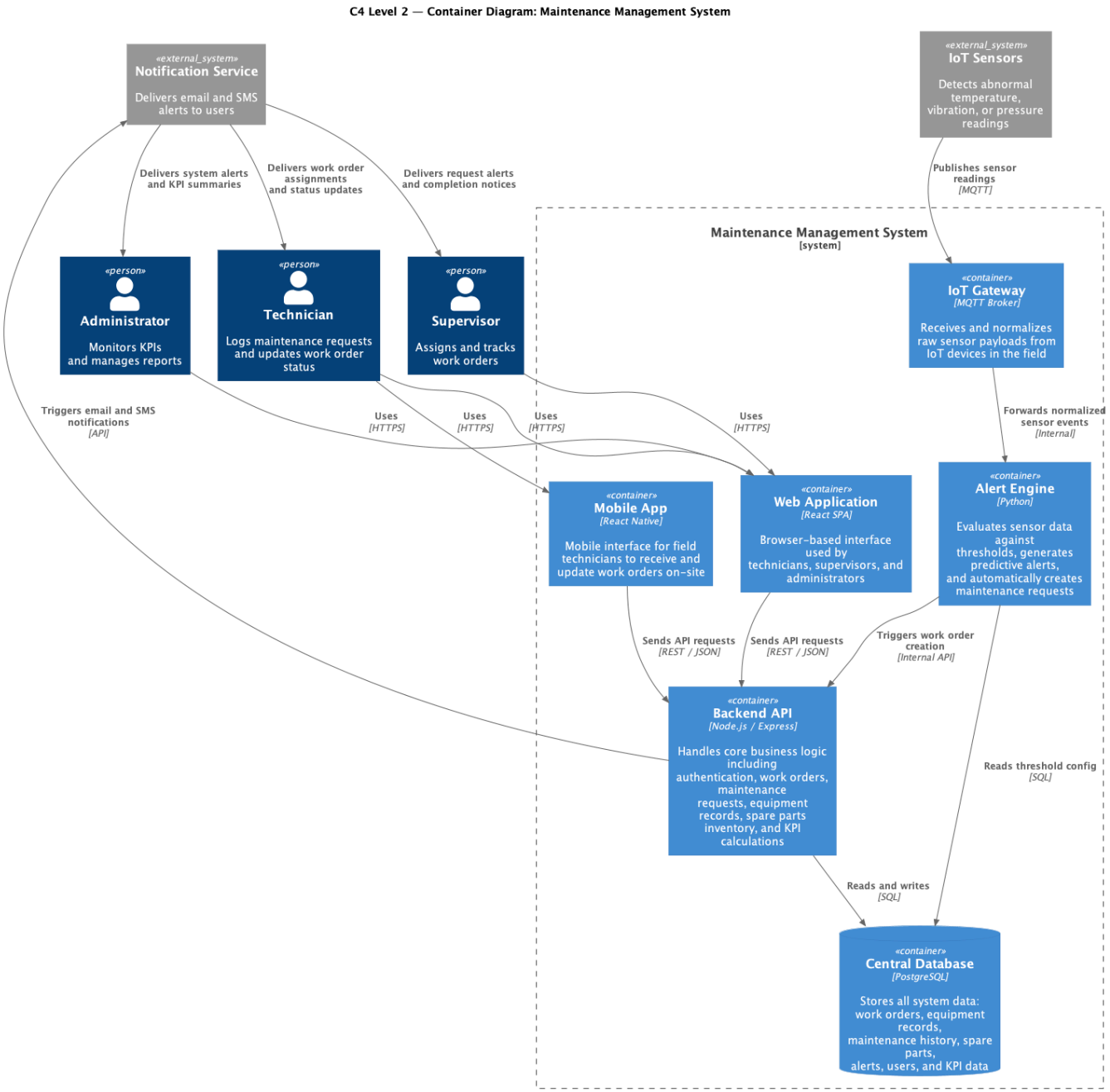
External Systems

- **IoT Sensors:** Provide abnormal readings related to equipment condition.
- **Notification Service:** Sends alerts and assignment notifications to supervisors and technicians.

The context model establishes the overall system boundary and shows how the MMS acts as the central platform coordinating maintenance operations between human users and external services.

2.2 Container Model

The C4 Level 2 Container Diagram decomposes the Maintenance Management System into its main technical containers and shows how they communicate to support the maintenance workflow.



The container model shows that the Maintenance Management System consists of the following main containers:

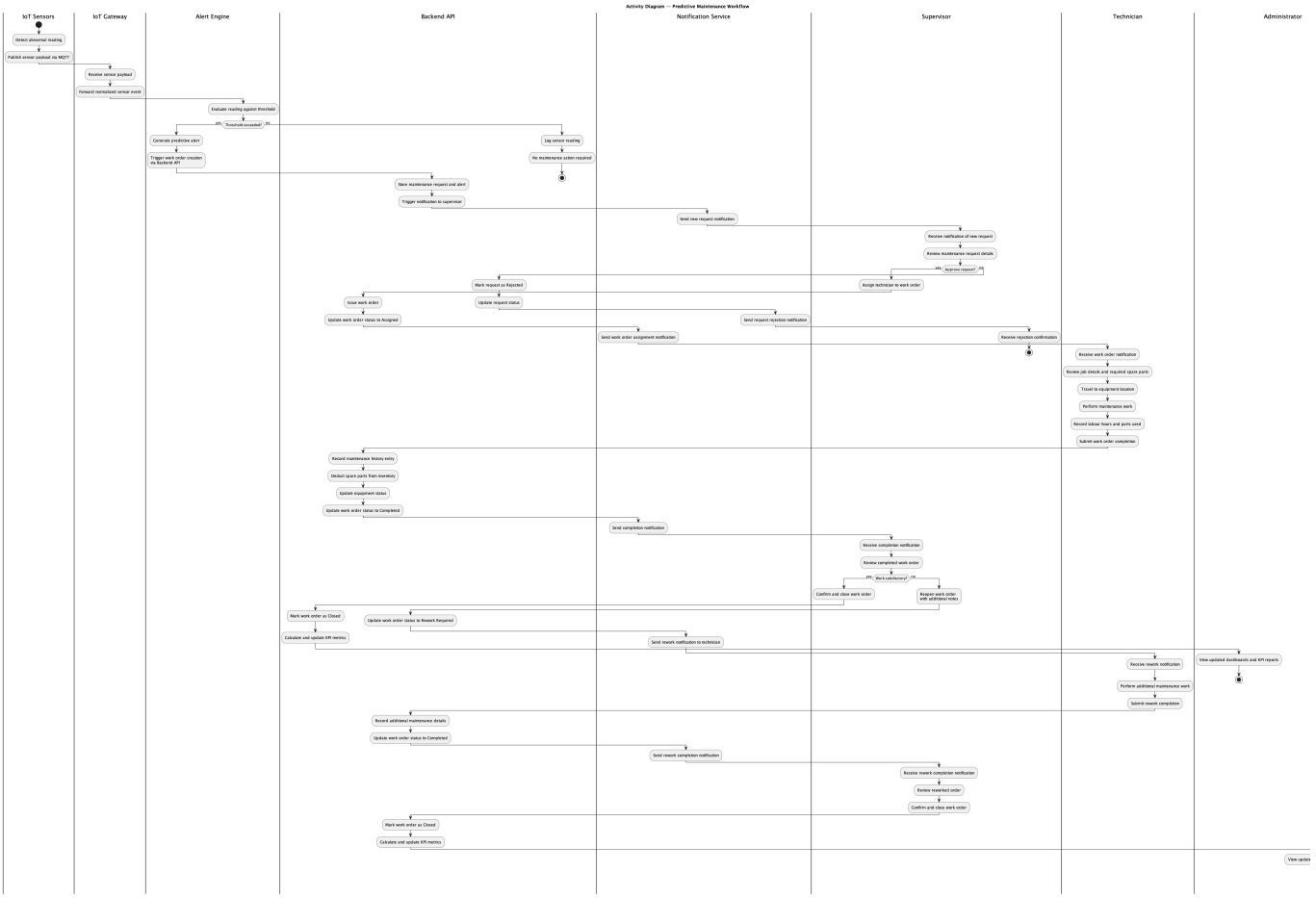
- **Web Application:** A browser-based interface used by technicians, supervisors, and administrators.
- **Mobile App:** A mobile interface mainly used by field technicians to receive and update work orders on-site.
- **Backend API:** Handles the main business logic, including authentication, maintenance requests, work orders, equipment records, spare parts inventory, and KPI calculations.
- **Central Database:** Stores work orders, equipment records, maintenance history, spare parts, alerts, users, and KPI data.
- **IoT Gateway:** Receives raw sensor payloads from IoT sensors using MQTT and normalizes them.
- **Alert Engine:** Evaluates sensor readings against thresholds, generates predictive alerts, and triggers automatic maintenance request creation.
- **Notification Service:** Sends email and SMS notifications to technicians, supervisors, and administrators.

The diagram shows that technicians, supervisors, and administrators interact with the system through the web application, while technicians may also use the mobile application in the field. Both applications communicate with the Backend API using REST/JSON. The Backend API reads and writes data to the central database and triggers the Notification Service when important workflow events occur.

The IoT part of the system is separated into the IoT Gateway and Alert Engine. IoT sensors publish readings to the IoT Gateway using MQTT. The gateway forwards normalized events to the Alert Engine, which checks readings against threshold configurations and triggers the Backend API when a predictive maintenance request must be created.

2.3 Activity Diagram

The activity diagram illustrates the predictive maintenance workflow with swimlanes representing the main participants in the process.



Key Activities

1. IoT sensors send abnormal sensor readings.
2. The system analyzes the readings and generates a predictive alert.
3. A predictive maintenance request is automatically created.
4. The supervisor is notified and reviews the request.
5. If approved, a technician is assigned.
6. The technician performs the maintenance work and submits completion.
7. The supervisor reviews the result and either closes the work order or requests rework.
8. The administrator monitors the updated KPIs and dashboards.

This activity diagram describes the end-to-end predictive maintenance workflow in the Maintenance Management System. The process begins when IoT sensors detect an abnormal reading and forward it to the system. The system evaluates the reading, creates a predictive maintenance request, and notifies the supervisor. After the supervisor reviews and approves the request, a technician is assigned to perform the maintenance task. Once the task is completed, the technician submits the work results, and the supervisor either confirms closure or returns the work for reprocessing. The process ensures that predictive maintenance requests are handled systematically and that maintenance performance is reflected in the system dashboards.

---

3. Use Case Models and Interaction Design

3.1 Selected Primary Actors

The primary actors selected from the context model are:

- **Technician**
- **Supervisor**
- **Administrator**
- **IoT Sensors**

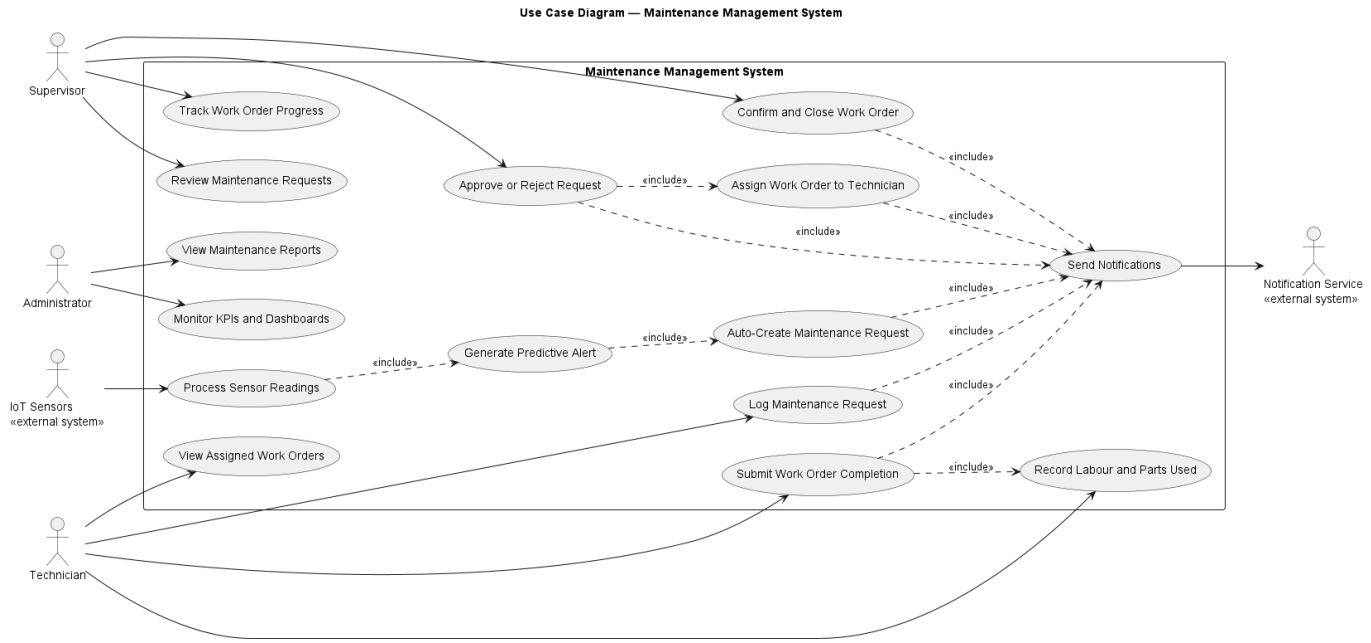
From the context diagram, these actors interact with the Maintenance Management System to perform the core maintenance operations. Based on these interactions, the following UML artefacts have been produced:

- **Composite use case diagram** for the Maintenance Management System
- **Sequence diagram – High-Level (Stakeholders)**
- **Sequence diagram – Detailed (Developers)**

These diagrams are consistent with the context view, which shows that technicians, supervisors, administrators, and IoT sensors all interact with MMS in different ways to support maintenance workflows.

3.2 Composite Use Case Diagram

The composite use case diagram shows the main use cases supported by the Maintenance Management System.



The use case diagram shows that the system supports:

- Logging maintenance requests
- Viewing assigned work orders
- Reviewing maintenance requests
- Approving or rejecting requests
- Assigning technicians
- Tracking work order progress
- Submitting work order completion
- Recording labour and parts used
- Sending notifications
- Monitoring dashboards and KPIs
- Processing sensor readings
- Generating predictive alerts
- Auto-creating maintenance requests

3.3 Use Case Descriptions

The tabular use case descriptions for the Maintenance Management System are provided in a separate file:

`docs/use_case_descriptions.md`

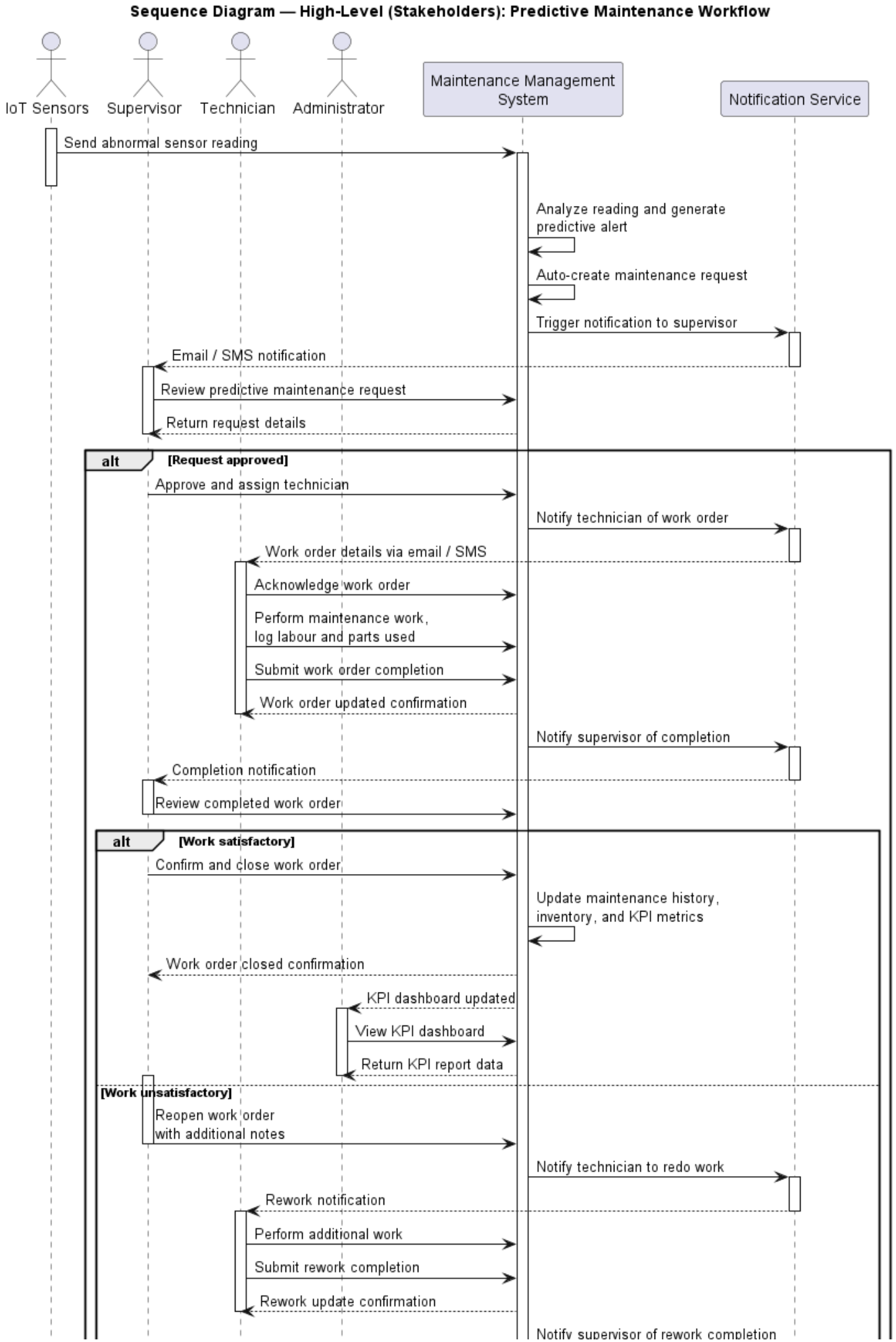
These descriptions define the use case name, actors, goal, preconditions, postconditions, main success scenario, and alternative flows for the main system functions.

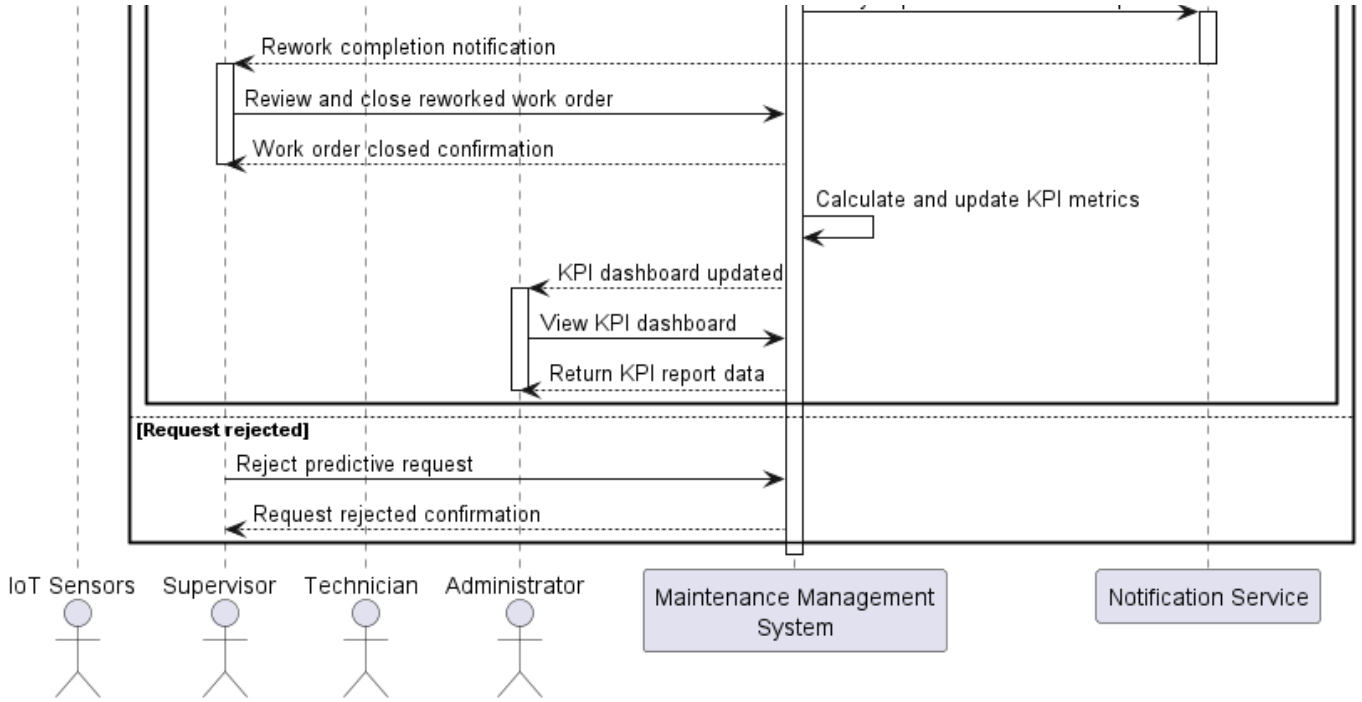
---

4. Sequence Diagrams

4.1 High-Level Sequence Diagram

This sequence diagram shows the stakeholder-level interaction for the predictive maintenance workflow.


Sequence Diagram — High-Level (Stakeholders): Predictive Maintenance Workflow
IoT Sensors
Supervisor
Technician
Administrator
Maintenance Management System
Notification Service
Send abnormal sensor reading
Analyze reading and generate predictive alert
Auto-create maintenance request
Trigger notification to supervisor
Email / SMS notification
Review predictive maintenance request
Return request details
alt
[Request approved]
Approve and assign technician
Notify technician of work order
Work order details via email / SMS
Acknowledge work order
Perform maintenance work, log labour and parts used
Submit work order completion
Work order updated confirmation
Notify supervisor of completion
Completion notification
Review completed work order
alt
[Work satisfactory]
Confirm and close work order
Update maintenance history, inventory, and KPI metrics
Work order closed confirmation
KPI dashboard updated
View KPI dashboard
Return KPI report data
[Work unsatisfactory]
Reopen work order with additional notes
Notify technician to redo work
Rework notification
Perform additional work
Submit rework completion
Rework update confirmation
Notify supervisor of rework completion



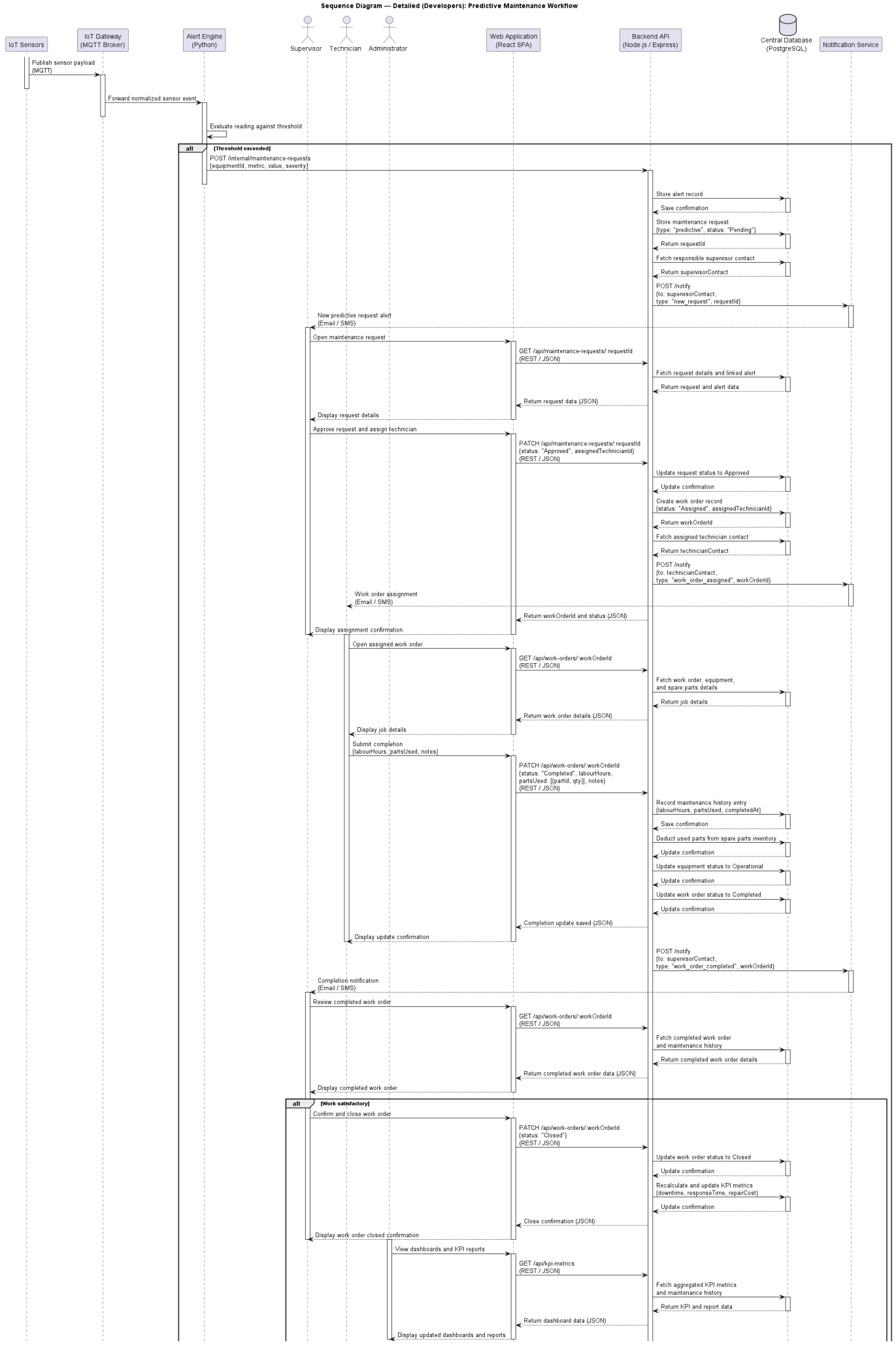
Key Interactions

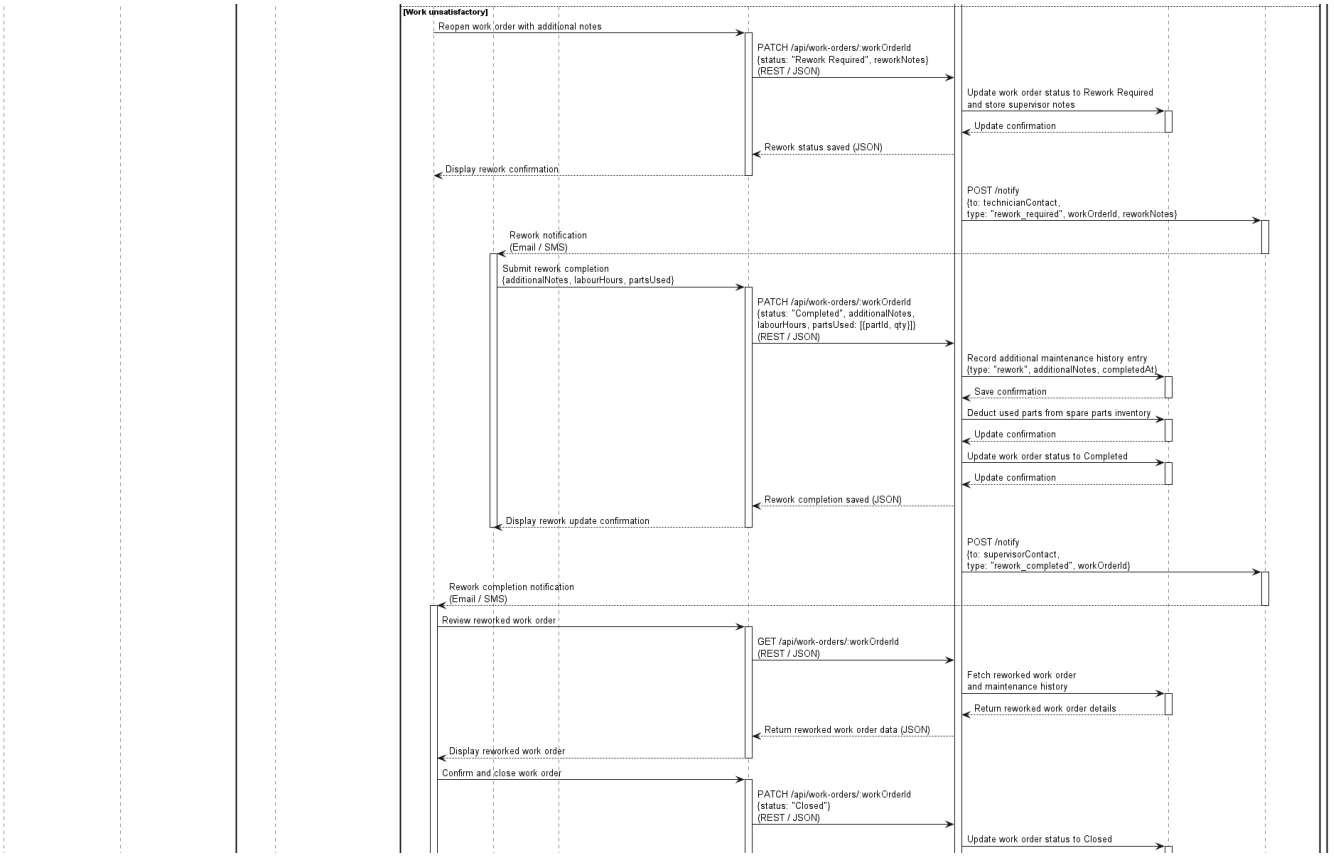
1. IoT sensors send abnormal sensor readings to the system.
2. The system analyzes the reading and creates a predictive maintenance request.
3. The supervisor is notified of the new request.
4. The supervisor reviews the request and approves it.
5. The system assigns the work order to a technician.
6. The technician performs the maintenance and submits completion.
7. The supervisor reviews the result and closes the work order.
8. The administrator views updated KPI dashboards.

This high-level sequence diagram presents the predictive maintenance workflow from a business and stakeholder perspective. It emphasizes the interactions between IoT sensors, the Maintenance Management System, the supervisor, the technician, and the administrator. The diagram captures the major workflow events without focusing on internal implementation details.

4.2 Detailed Sequence Diagram

The detailed sequence diagram provides the developer-level interaction of the Maintenance Management System. It shows the internal communication between the technical components involved in handling predictive maintenance requests.


Sequence Diagram — Detailed (Developers): Predictive Maintenance Workflow
IoT Sensors
IoT Gateway (MQTT Broker)
Alert Engine (Python)
Supervisor
Technician
Administrator
Web Application (React SPA)
Backend API (Node.js / Express)
Central Database (PostgreSQL)
Notification Service
Publish sensor payload (MQTT)
Forward normalized sensor event
Evaluate reading against threshold
alt
[Threshold exceeded]
POST /internal/maintenance-requests {equipmentId, metric, value, severity}
Store alert record
Save confirmation
Store maintenance request {type: "predictive", status: "Pending"}
Return requestId
Fetch responsible supervisor contact
Return supervisorContact
POST /notify {to: supervisorContact, type: "new_request", requestId}
New predictive request alert (Email / SMS)
Open maintenance request
GET /api/maintenance-requests/:requestId (REST / JSON)
Fetch request details and linked alert
Return request and alert data
Return request data (JSON)
Display request details
Approve request and assign technician
PATCH /api/maintenance-requests/:requestId {status: "Approved", assignedTechnicianId} (REST / JSON)
Update request status to Approved
Update confirmation
Create work order record {status: "Assigned", assignedTechnicianId}
Return workOrderId
Fetch assigned technician contact
Return technicianContact
POST /notify {to: technicianContact, type: "work_order_assigned", workOrderId}
Work order assignment (Email / SMS)
Return workOrderId and status (JSON)
Display assignment confirmation
Open assigned work order
GET /api/work-orders/:workOrderId (REST / JSON)
Fetch work order, equipment, and spare parts details
Return job details
Return work order details (JSON)
Display job details
Submit completion {labourHours, partsUsed, notes}
PATCH /api/work-orders/:workOrderId {status: "Completed", labourHours, partsUsed: [{partId, qty}], notes} (REST / JSON)
Record maintenance history entry {labourHours, partsUsed, completedAt}
Save confirmation
Deduct used parts from spare parts inventory
Update confirmation
Update equipment status to Operational
Update confirmation
Update work order status to Completed
Update confirmation
Completion update saved (JSON)
Display update confirmation
POST /notify {to: supervisorContact, type: "work_order_completed", workOrderId}
Completion notification (Email / SMS)
Review completed work order
GET /api/work-orders/:workOrderId (REST / JSON)
Fetch completed work order and maintenance history
Return completed work order details
Return completed work order data (JSON)
Display completed work order
alt
[Work satisfactory]
Confirm and close work order
PATCH /api/work-orders/:workOrderId {status: "Closed"} (REST / JSON)
Update work order status to Closed
Update confirmation
Recalculate and update KPI metrics {downtime, responseTime, repairCost}
Update confirmation
Close confirmation (JSON)
Display work order closed confirmation
View dashboards and KPI reports
GET /api/kpi-metrics (REST / JSON)
Fetch aggregated KPI metrics and maintenance history
Return KPI and report data
Return dashboard data (JSON)
Display updated dashboards and reports



The detailed sequence diagram includes interactions between:

- **IoT Sensors**
- **IoT Gateway / MQTT Broker**
- **Alert Engine**
- **Web Application**
- **Backend API**
- **Central Database**
- **Notification Service**
- **Supervisor**
- **Technician**
- **Administrator**

This developer-level diagram explains how sensor data flows through the system, how abnormal readings are evaluated, how predictive maintenance requests are created and stored, how notifications are generated, and how user actions update the work order lifecycle. Compared with the stakeholder-level diagram, this version provides more technical detail about the system internals, database operations, and notification flow.
